

RESULTS

serve-sim + simstream versus stock serve-sim

We replaced *serve-sim*'s video path with the *simstream* engine and kept *serve-sim*'s UI, input and tools. Then we tested how its video should travel: WebSocket or WebRTC.

2026-09-25, UPDATED 2026-10-01. FORK: `expo-device-hub-simstream`, BRANCH `simstream-video`, TRANSPORT SWITCH IN PR #218.

Summary

4.2 × the pixels of stock WebRTC, at the same latency	59.8_{fps} distinct frames shown, against 48–51 for WebRTC	0 freezes over 50 ms in 160 s of measurement	~7 × lower latency than <i>serve-sim</i> 's HTTP/AVCC mode
---	--	--	--

Against *serve-sim*'s own two video modes, the *simstream* path:

- **Sends 3.5–4.2× the pixels at the same latency as WebRTC.** It streams at full native resolution (1206×2622 on the iPhone 16 Pro, 1320×2868 on the 17 Pro Max). Stock WebRTC streams at 640×1392, and its latency is still no lower.
- **Holds 60 fps.** In the controlled benchmark it showed 59.8–59.9 distinct frames per second and skipped 0.2% of frames. Stock WebRTC showed 48–51 fps and skipped 19–26%.
- **Never froze.** There were 0 freezes over 50 ms in 160 s of measurement, against 20 for WebRTC and 32 for HTTP/AVCC.
- **Has about 7× lower latency than *serve-sim*'s HTTP/AVCC mode** (38–42 ms against 273–283 ms).
- **Found a *serve-sim* bug that affects every mode proxied through Node.** The fix is included.

Where it **didn't** win: the over-the-network route recording shows *simstream* and WebRTC tied on touch-to-screen time and on frames shown. On latency alone, the two are at parity; the difference is what each delivers at that latency (resolution, pacing, no freezes).

How simstream’s video should travel (sections 6–11, 2026-09-29 to 10-01):

6_x the pans per second over WebRTC (RTP) at 1% packet loss: 22.8 against 3.8 over WebSocket	183_{ms} worst freeze over RTP at 1% loss, against 622 ms over WebSocket	21 % less smooth: what RTP gives up on a clean link (40.5 against 51.4 pans per second)	2 transports, switchable per viewer in Agent Hub (PR #218)
--	--	---	--

- **On a clean link, WebSocket wins.** It is about 20% smoother than a WebRTC video track (RTP), and equally responsive.
- **Under packet loss, WebSocket collapses and RTP degrades.** At 1% loss each way, WebSocket falls to 4–6 new pictures a second with freezes over half a second. RTP holds 23–26.
- **Two causes behind the collapse.** TCP holds back every frame behind a lost packet, and the engine’s rate control reads that delay as congestion and cuts to its floor. With rate control taken out, WebSocket still reaches only half of RTP’s smoothness at 1% loss.
- **A WebRTC data channel is not a contender.** It collapsed on a clean link in every run.
- **The earlier lead of Agent Hub over the engine page came from its touch input path, not its transport.** With touches injected the same way, the two match.
- **Recommendation:** keep WebSocket as the default, offer RTP for lossy or remote viewers (done, PR #218), and make the engine’s rate control loss-aware. That last step is the largest remaining gain for both transports.

Method: measure the engine first, add the network second. The comparison runs in two stages, deliberately. Stage one puts viewer and server on the same machine, so there is no network at all and any difference in latency belongs to the video path. It also allows an absolute measurement: the barcode clock and the viewer share a clock, so frame age is real milliseconds rather than an inference. Stage two adds a real network. That stage costs precision — across two machines the clocks differ, so frame age cannot be measured at all, and the coarser touch-to-screen metric replaces it. Starting there would have folded a 13 ms path and its jitter into every number, with no way to separate engine from transport.

SECTION 1

Controlled latency benchmark (same machine)

MACHINE M4 Max, iPhone 16 Pro simulator on iOS 18.6.

SCENE a barcode clock page in the simulator. Headless Chrome decodes the barcode from whatever the viewer page displays and records how old the displayed frame is.

RUNS 4 runs of 20 s per mode and scene, taken one at a time under this protocol:

1. Wait for machine load to settle.
2. Pick a mode at random and take a single run.
3. Repeat, so the modes stay intermingled — never 20 consecutive runs of one mode and then the next.

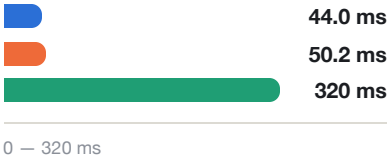
The best run per mode and scene is the one kept.

HEAVY SCENE — THREE METRICS, THREE MODES

simstream WebRTC HTTP/AVCC

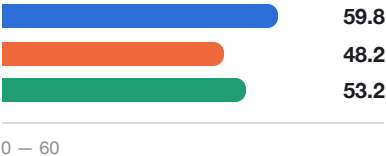
FRAME AGE, P95

milliseconds · lower is better



DISTINCT FPS SHOWN

frames per second · higher is better



SOURCE FRAMES SKIPPED

percent · lower is better

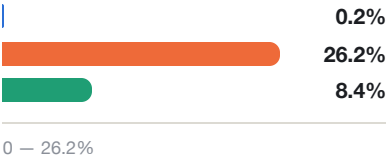


TABLE 1 · LIGHT SCENE

METRIC	• SERVE-SIM + SIMSTREAM	STOCK WEBRTC	STOCK HTTP/AVCC
Frame age, mean ↓	37.6 ms	36.0 ms	272.6 ms
Frame age, p95 ↓	43.5 ms	44.8 ms	298 ms
Distinct fps shown ↑	59.9	50.6	54.1
Source frames skipped ↓	0.2%	18.6%	6.0%
Freezes over 50 ms (80 s total) ↓	0	12	16
Worst gap ↓	34 ms	68 ms	257 ms
Resolution ↑	1206×2622	640×1392	1206×2622

↓ lower is better · ↑ higher is better. Tinted column is the simstream path.

TABLE 2 · HEAVY SCENE

METRIC	• SERVE-SIM + SIMSTREAM	STOCK WEBRTC	STOCK HTTP/AVCC
Frame age, mean ↓	41.6 ms	38.8 ms	283.2 ms
Frame age, p95 ↓	44.0 ms	50.2 ms	320 ms
Distinct fps shown ↑	59.8	48.2	53.2
Source frames skipped ↓	0.2%	26.2%	8.4%
Freezes over 50 ms (80 s total) ↓	0	8	16
Worst gap ↓	50 ms	68 ms	316 ms

↓ lower is better · ↑ higher is better. Tinted column is the simstream path.

WebRTC’s mean latency is 1.6–2.8 ms lower. It gets there by encoding a quarter of the pixels, and it drops a fifth to a quarter of the frames. simstream’s p95 matches or beats it.

Engine timing (simstream at full resolution): capture to encoder 1.3 ms, encode 8.4 ms, network queue about 1 ms.

SECTION 2

Over the network

With the video paths separated on one machine, the same comparison was repeated across a real network. The engine numbers above are what this stage is measured against.

SERVER	serve-sim on the Mac Mini (M4 Pro, iPhone 17 Pro Max on iOS 26.5).
VIEWER	headless Chrome on an off-prem MacBook, about 13 ms away over a direct Tailscale path.
ROUTE	the same scripted 24-input route (Settings → General → About → back → home → Calendar year/month/day/multi-day). Every touch was sent from the viewer page, so video and input both crossed the network.
MEASUREMENT	touch to screen is the time from the viewer dispatching a touch to the viewer’s screen first changing. It includes the app’s own reaction time, which is identical across modes.

TABLE 3 · TOUCH TO SCREEN OVER THE NETWORK

METRIC	• SERVE-SIM + SIMSTREAM	STOCK WEBRTC	STOCK HTTP/AVCC
Touch to screen, median ↓	100 ms	100 ms	300 ms
Touch to screen, mean ↓	101 ms	97 ms	286 ms
Touch to screen, p90 ↓	183 ms	183 ms	417 ms
Distinct frames in the 1 s after each touch (sum of 24) ↑	868	862	698
Resolution ↑	1320×2868	640×1392	1320×2868
Recorder stalls ↓	0	0	0

↓ lower is better · ↑ higher is better. Tinted column is the simstream path.

- **Network result:** simstream and WebRTC are tied on responsiveness and smoothness. simstream carries 4.2× the pixels while doing it. HTTP/AVCC is about 3× slower to respond.
- **On the same machine** (the same route on the M4), simstream showed 838 frames in the 1 s after touches, against 754 for WebRTC and 709 for AVCC.

SECTION 3

Why: the architectural differences

- **Capture is locked to the simulator’s render clock.** simstream encodes a frame when the simulator produces one (damage callbacks on the framebuffer surface). serve-sim’s WebRTC path samples the framebuffer on its own timer, which drifts against the simulator’s 60 Hz. Some frames are captured twice and others never, hence the 19–26% skipped.
- **Nothing queues.** simstream has one encoder per viewer, and the viewer acks every frame. Bitrate control is delay-based, from those acks, and a stall pauses encoding instead of piling frames up. serve-sim’s HTTP/AVCC path has a 16-frame native queue that stays full, which is about 270 ms of standing latency.
- **Full resolution with no downscaling.** VideoToolbox low-latency H.264 encodes the full frame in about 8 ms. Upstream caps H.264 over WebRTC at 1280 on the long edge; per its own source, full resolution “stalls and does not recover”. In practice, the stream came out at 640×1392.
- **Presentation is paced per refresh.** Decoded frames are drawn immediately, at most one per display refresh. A burst after a hiccup collapses to the newest frame instead of fast-forwarding.

SECTION 4

The serve-sim bug we found

FIX INCLUDED · COMMIT 6512155

serve-sim's `/grid/api/memory` endpoint is polled by its UI every 5 s. It ran `ps -axo rss=,args=`, `sysctl` and `vm_stat` with `execSync`, blocking Node's event loop for about 75 ms each time. Every stream proxied through the process (HTTP/AVCC, MJPEG, and our socket) froze for that long every 5 s. Switching to async `execFile` (commit 6512155) took simstream through serve-sim from 16 freezes to 0 per 80 s of measurement. That matches simstream standalone, and the fix applies upstream on its own.

SECTION 5

What produced the gains

Each change below is paired with the measurement that supports it, so the approaches can be picked up and investigated one at a time. Only the last one was measured as an isolated before/after on the same build. The other four are design differences between the two video paths, so their effects are attributed from the mode-to-mode comparison, not from an ablation.

TABLE 4 · WHAT PRODUCED THE GAINS

CHANGE	MEASURED EFFECT	EVIDENCE
Capture on framebuffer damage callbacks instead of sampling on an independent timer	Source frames skipped 26.2% → 0.2%; distinct frames shown 48.2 → 59.8 fps	Heavy scene, simstream against stock WebRTC
One encoder per viewer, the viewer acking every frame, delay-based bitrate control from those acks, and a stall that pauses encoding	Removes the 16-frame native queue holding about 270 ms of standing latency: frame age mean 283.2 ms → 41.6 ms	Heavy scene, simstream against stock HTTP/AVCC
VideoToolbox low-latency H.264 over the full frame with no downscale, at about 8.4 ms per encode	1320×2868 instead of 640×1392, 4.2× the pixels per frame, with touch to screen unchanged at a 100 ms median	Over-the-network route; engine timing
Draw each decoded frame immediately, at most one per display refresh, collapsing a post-hiccup burst to the newest frame	Worst gap 50 ms against 68 ms for WebRTC, with no fast-forwarding after a hiccup	Heavy scene
<code>execSync</code> → async <code>execFile</code> in <code>/grid/api/memory</code> (commit 6512155)	Freezes over 50 ms: 16 → 0 per 80 s, for every mode proxied through Node	Isolated before/after on the same build

Not measured: neither bandwidth nor memory was instrumented in this round, so no claim is made for either. The full-resolution path necessarily sends more bytes than the 640×1392 WebRTC stream; how many more is open, and it is the first thing worth measuring next.

Transport: the six-way comparison

With the engine settled, the open question was how its video should travel. Six approaches played the same scripted Maps route (pans, one-finger zooms, flings and a jiggle, with the finger moving at 60 Hz), one at a time, with Maps reset identically before each.

SERVER	a MacBook in Southampton running the simulator (iOS 26.3).
VIEWER	headless Chrome on the Mac Mini, over a direct Tailscale path.
MEASUREMENT	new pictures per second while the finger moves (frames whose screen area differs from the previous one), fling momentum in the half second after the lift, and touch to picture. Touch to picture includes Maps’ own reaction, the same in every pane.

TABLE 5 · FIRST PASS, ONE RUN EACH, NEW PICTURES PER SECOND

APPROACH	PANS ↑	ZOOM ↑	FLING MOMENTUM ↑	TOUCH TO PICTURE, MEDIAN / WORST ↓
1 · simstream + Agent Hub (WebSocket)	48.6	35.6	58–60	158 / 217 ms
4 · simstream engine page (WebSocket)	35.7	35.2	28–60	142 / 233 ms
5 · simstream + WebRTC data channel	34.3	28.4	52–60	200 / 300 ms
6 · simstream + WebRTC video track (RTP)	31.8	28.8	49–54	167 / 250 ms
2 · stock WebRTC	32.5	17.6	31–46	167 / 350 ms
3 · stock HTTP	38.6	22.8	22–60	392 / 467 ms

This didn’t settle the transport. Pane 1 sent touches through serve-sim and panes 4–6 through the engine, so its lead could come from input rather than transport; there was one run each; the WebRTC panes took the public internet while the WebSocket panes took the tailnet; and the conditions where WebRTC should win (packet loss) were untested. Sections 7–10 remove each of those in turn.

Equal conditions

MADE EQUAL	every simstream pane injects touches through the engine (pane 1 through a recorder preload), every pane uses the same direct tailnet path (the WebRTC bridges bind to it, with no STUN), one engine binary, and the same transition setting.
SERVER	the Southampton MacBook.
VIEWER	headless Chrome on the M4, about 18 ms away.
RUNS	3 rounds, each pane once per round, interleaved.

TABLE 6 · EQUAL CONDITIONS, CLEAN LINK, MEAN OF 3 [RANGE]

APPROACH	PANS ↑	ZOOM ↑	FLING MOMENTUM ↑	TOUCH TO PICTURE, MEDIAN / WORST ↓
1 · Agent Hub (WebSocket)	35.4 [32.5–37.9]	33.5	57.8 [56.5–59.0]	150 / 239 ms
4 · engine page (WebSocket)	37.3 [36.5–38.5]	33.5	57.8 [57.0–59.0]	142 / 250 ms
6 · WebRTC video track (RTP)	33.9 [32.5–35.7]	30.9	52.7 [52.1–53.7]	156 / 250 ms
5 · WebRTC data channel	9.1 [6.8–12.4]	7.8	18.4 [13.4–22.8]	225 / 450 ms

- **Pane 1’s lead was its input path.** With touches injected the same way, panes 1 and 4 match. Its 48.6 in the first pass came from serve-sim’s touch injection (moves are sent as touch-downs; the engine sends drags).
- **RTP trails WebSocket a little, consistently.** Its momentum was below every WebSocket run in every round.
- **The data channel collapsed in all three rounds.**

SECTION 8

Packet loss

LOSS	dropped at random on the viewer’s machine, each way, on all traffic with the server (both its tailnet addresses), so TCP and UDP take the same loss. A ping running alongside every run confirmed the loss was in effect.
WHY IT WORKS NOW	dummynet rules only apply in pf’s main ruleset. Loaded into a <code>com.apple</code> anchor, which is where earlier attempts put them, they’re accepted but never applied.
RUNS	panes 1, 4 and 6 at 0%, 1% and 2% loss, 3 interleaved rounds. Pane 1 injected touches through the engine, as in section 7.
WORST FREEZE	the longest stretch with no new picture while the picture should be moving (pans, jiggle, fling momentum).

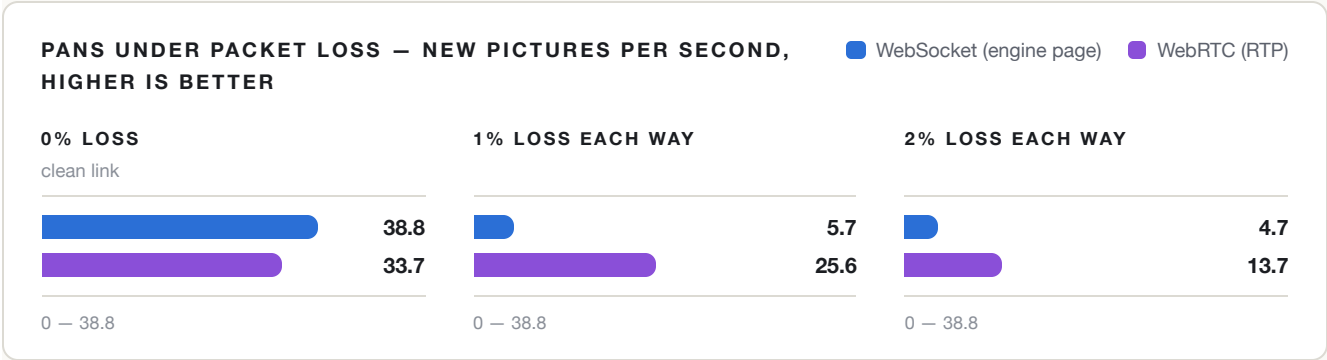


TABLE 7 · PACKET LOSS, MEAN OF 3

LOSS · APPROACH	PANS ↑	FLING MOMENTUM ↑	WORST FREEZE ↓	TOUCH TO PICTURE, MEDIAN ↓
0% · 1 Agent Hub (WebSocket)	39.4	58.8	89 ms	150 ms
0% · 4 engine page (WebSocket)	38.8	57.5	89 ms	153 ms
0% · 6 WebRTC (RTP)	33.7	51.4	94 ms	156 ms
1% · 1 Agent Hub (WebSocket)	4.7	9.8	511 ms	217 ms
1% · 4 engine page (WebSocket)	5.7	16.6	567 ms	239 ms
1% · 6 WebRTC (RTP)	25.6	32.7	133 ms	153 ms
2% · 1 Agent Hub (WebSocket)	3.9	12.0	506 ms	219 ms
2% · 4 engine page (WebSocket)	4.7	11.5	461 ms	256 ms
2% · 6 WebRTC (RTP)	13.7	8.8	272 ms	172 ms

At 1% loss WebSocket is unusable: about 5 new pictures a second and half-second freezes. RTP keeps working, and responds as fast as it does on a clean link. At 2% RTP degrades too.

SECTION 9

Why WebSocket collapses

The engine's log showed the WebSocket viewer's target bitrate pinned at its 1 Mbps floor under loss, with the encoder dropping most frames, while the RTP viewer held 5.5–10 Mbps at 50–60 fps. To separate the transport from the rate control, panes 4 and 6 were rerun with the bitrate pinned at 8 Mbps (`SIMSTREAM_FIXED_BITRATE`), 2 rounds.

TABLE 8 · BITRATE PINNED AT 8 MBPS, MEAN OF 2

LOSS · TRANSPORT	PANS ↑	FLING MOMENTUM ↑	WORST FREEZE ↓
0% · WebSocket	32.4	55.5	109 ms
0% · WebRTC (RTP)	33.0	52.2	100 ms
1% · WebSocket	13.6	17.6	367 ms
1% · WebRTC (RTP)	25.4	40.7	133 ms
2% · WebSocket	9.6	12.4	525 ms
2% · WebRTC (RTP)	22.9	30.8	217 ms

- **TCP causes part of it.** One lost packet holds back every frame behind it until it is resent. Without rate control, WebSocket still reaches only half of RTP's smoothness, with freezes of 0.4–0.5 s.

- **The rate control makes it much worse.** It is delay-based, so TCP's retransmission delay reads as congestion and the bitrate falls to the floor. Taking it out more than doubles WebSocket's smoothness at 1% loss. It is also what dragged RTP down at 2% (momentum 8.8 with it, 30.8 without).

SECTION 10

The switch in Agent Hub (PR #218)

Agent Hub now lets each viewer pick the transport: Stream settings → simstream transport, or `?simstream=websocket|rtp`. WebSocket stays the default. The RTP bridge runs in serve-sim's simstream relay process, off the main event loop, and the engine is unchanged.

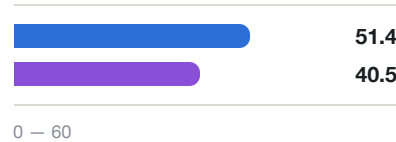
BUILD	the PR's build, packed and run on the Southampton MacBook (<code>--transport http --codec simstream</code>).
INPUT	serve-sim's own, as shipped, in both panes. The panes differ only in the switch.
RUNS	0%, 1% and 2% loss, 3 interleaved rounds, viewer on the M4.

AGENT HUB WITH THE SWITCH — WEBSOCKET AGAINST WEBRTC (RTP)

WebSocket WebRTC (RTP)

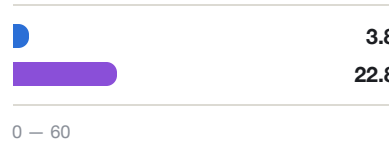
PANS, CLEAN LINK

new pictures per second · higher is better



PANS, 1% LOSS

new pictures per second · higher is better



WORST FREEZE, 1% LOSS

milliseconds · lower is better

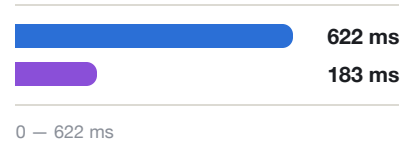


TABLE 9 · AGENT HUB WITH THE SWITCH, MEAN OF 3 [RANGE]

LOSS · TRANSPORT	PANS ↑	FLING MOMENTUM ↑	WORST FREEZE ↓	TOUCH TO PICTURE, MEDIAN ↓
0% · WebSocket	51.4 [50.0–52.1]	58.6 [58.2–59.0]	72 ms	147 ms
0% · WebRTC (RTP)	40.5 [34.3–44.6]	49.1 [48.6–50.2]	94 ms	153 ms
1% · WebSocket	3.8 [3.6–4.0]	16.1 [13.4–18.4]	622 ms	225 ms
1% · WebRTC (RTP)	22.8 [18.6–27.9]	24.2 [14.2–42.2]	183 ms	164 ms
2% · WebSocket	4.6 [3.6–6.1]	13.2 [12.0–14.8]	545 ms	228 ms
2% · WebRTC (RTP)	8.4 [6.8–10.4]	9.6 [7.7–11.7]	578 ms	164 ms

Every run landed all 21 scripted inputs, and every RTP run used a direct tailnet path (17–26 ms). With serve-sim's own input, WebSocket's clean-link lead is larger than in section 7 (51.4 against 40.5): its touch injection produces more frames, for both transports.

SECTION 11

Recommendations

- **Keep WebSocket as the default** for local and clean links: it is the smoothest there and the simplest to run (one TCP connection through any HTTPS proxy, no NAT traversal, works in Electron).
- **Offer RTP for lossy or remote viewers.** Done in PR #218. It needs `--stun-url / --turn-url` to reach viewers across NAT, and it can't play in Electron, whose WebRTC lacks H.264. Decoding the RTP frames with WebCodecs instead of a `<video>` element would lift that.
- **Make the engine's rate control loss-aware.** For WebSocket, stop reading one-off retransmission delay as congestion; for RTP, take loss from the browser's receiver reports; consider a higher floor. It is the largest remaining gain for both transports.
- **Match serve-sim's touch injection in the engine.** serve-sim sends moves as touch-downs and the engine sends drags; serve-sim's way produced more frames in every comparison.
- **Retire the data channel.**

SECTION 12

Caveats

- **Machine load:** a run started only once load had settled, which is not the same as idle. In sections 1–2 the M4 was shared (other agents, Docker), and two heavy reps began at a load around 70. In sections 7–10 the M4 was the viewer and ran at a load of 40–130; every pane saw that range and no link between load and the scores showed up. Picking or rotating the next mode spreads residual load across all modes alike.
- **Stock WebRTC resolution:** the 640×1392 is upstream's default behavior at commit `ec75fe7`. The published 0.3.4 release ran WebRTC at 1206×2622 or 804×1748 in our earlier test, with similar latency and similar frame skipping (18%).
- **Network latency:** the barcode latency can't be measured across machines (the two clocks differ), so the network comparisons use touch to screen from the recordings. That's resolved to one frame (16.7 ms) and includes the app's reaction time.
- **Sample size:** the section 2 route is one recording per mode (24 touches each), and section 6 one run per approach. The latency benchmark is 4×20 s per mode and scene; sections 7, 8 and 10 are 3 rounds each, section 9 two. A rerun moves a result by about ± 1 –3 new pictures a second.
- **Loss model:** random, independent loss on one path of about 18 ms. Real loss comes in bursts, and longer paths make TCP's stalls worse, so the gap under real loss is likely larger. Bandwidth caps were not tested: the Maps scene uses about 1 Mbps at rest.